4-bit Array Multiplier: Parts A and B Report

Oct 19, 2026 · @Akshay

Team and scope

Parts A and B are complete: the RTL passes all 256 vectors, the testbench catches an injected fault, and the synthesized SKY130 netlist (61 cells, 756.976 µm²) passes the same 256 vectors at gate level.

Name	Roll number	Responsibility
Sale Sangameshwar	2024102017	Part A (RTL design and verification), Part B (logic synthesis)
Palla Rithik Reddy	2024102005	Part C (static timing analysis), Part D (physical design)

Item	Version / value
Environment	IIC-OSIC-TOOLS Docker image `hpretl/iic-osic-tools:2026.10`, run under WSL 2 on Windows 11
Simulator	Icarus Verilog 14.0 (devel)
Synthesis	Yosys 0.70
Process / library	SkyWater SKY130, `sky130_fd_sc_hd`, typical corner `tt_025C_1v80`
Repository	rithikreddypalla/DVD_RTL_to_GDS

Part A: RTL design

The multiplier registers its inputs and output, builds the product from 16 AND gates and 12 `full_adder` instances in three ripple-carry rows, and uses no `*` operator.

`rtl/full_adder.v` uses two `assign` statements:

```
\text{sum} = a \oplus b \oplus c_{in} \qquad c_{out} = ab + a\,c_{in} + b\,c_{in}
```

`rtl/mult_array.v` follows the Section 2 connection table exactly:

Stage	Inputs	Outputs
Partial products	pp*i*[j] = a_r[j] AND b_r[i]	16 AND gates
Row 0	pp0	p[0] = pp0[0]; r0 = {0, pp0[3], pp0[2], pp0[1]}
Row 1 (fa1_0 to fa1_3)	pp1[j], r0[j], c1[j], with c1[0] = 0	p[1] = s1[0]; r1 = {c1[4], s1[3], s1[2], s1[1]}
Row 2 (fa2_0 to fa2_3)	pp2[j], r1[j], c2[j], with c2[0] = 0	p[2] = s2[0]; r2 = {c2[4], s2[3], s2[2], s2[1]}
Row 3 (fa3_0 to fa3_3)	pp3[j], r2[j], c3[j], with c3[0] = 0	p[3] = s3[0]; p[7:4] = {c3[4], s3[3], s3[2], s3[1]}

One `always @(posedge clk or negedge rst_n)` block holds all 16 flip-flops: `a_r[3:0]` and `b_r[3:0]` (8) capture the inputs at clock edge 1, and `p[7:0]` (8) captures the array result at edge 2. An active-low `rst_n` clears all of them asynchronously. Latency is therefore 2 clock cycles, with one new product per cycle.

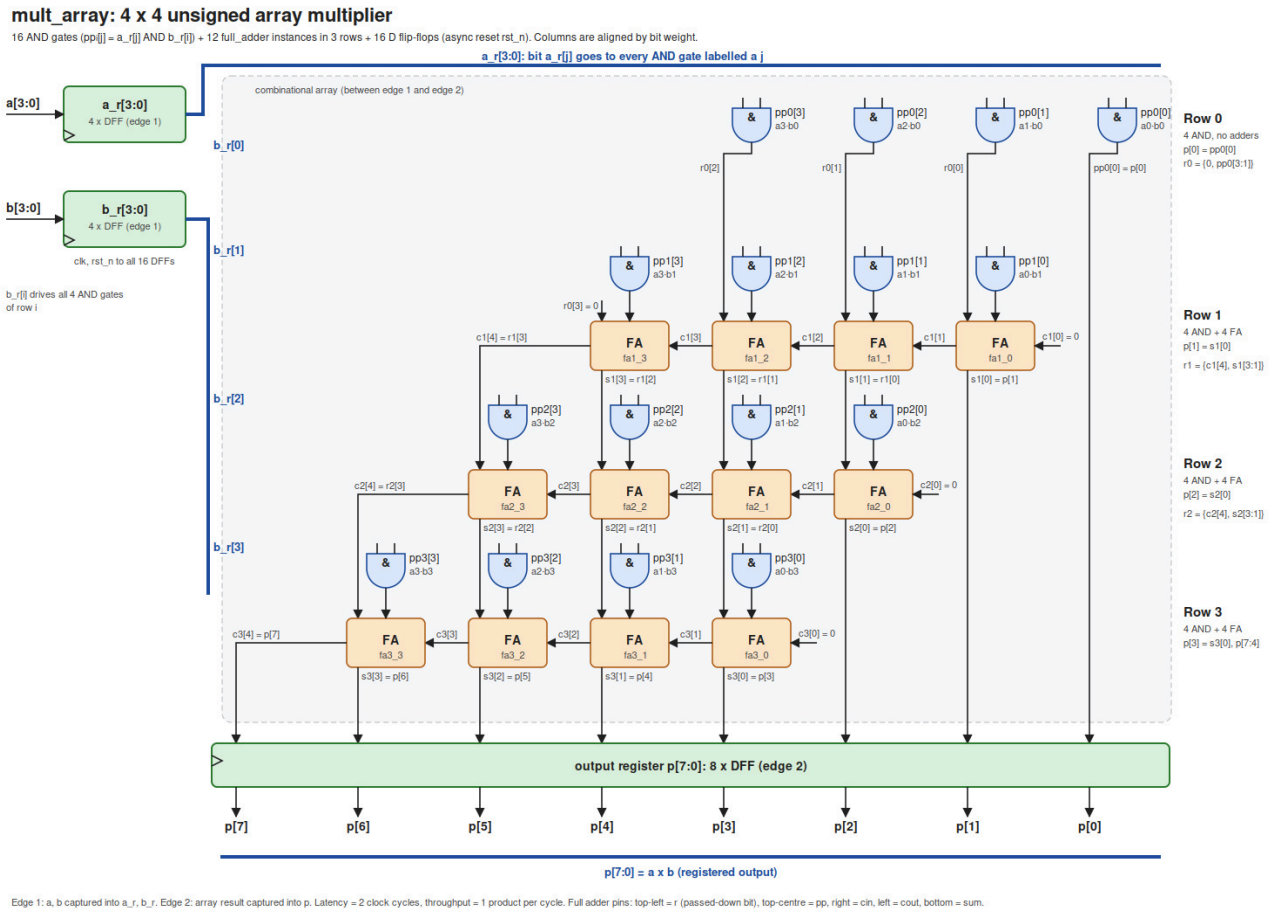


Figure 1: Block diagram of `mult_array`. Columns are aligned by bit weight, so each passed-down bit (r0, r1, r2) is a vertical wire; carries ripple right to left inside each row.

Part A: Verification

The RTL passes all 256 input combinations, and the testbench detects a deliberately broken full adder with 95 mismatches.

Testbench method (`tb/tb_mult.v`):

- 10 ns clock; `rst_n` held low for the first 2 cycles (0–20 ns), with a check that `p = 0` during reset.
- A new (a, b) pair is applied on every falling edge, for all 256 pairs (vector *i*: a = i[7:4], b = i[3:0]).
- Each vector is checked exactly 2 cycles after it is applied against the expected value `a * b` (the `*` operator appears only in the testbench). A design with latency 1 or 3 would fail.
- `!==` is used, so X or Z on `p` counts as an error. One line is printed per mismatch, then one summary line; the waveform is written to `mult.vcd`.

Run	Summary line
Correct RTL	`PASS: 256 vectors checked, 0 errors`
Fault injected: `assign sum = a ^ b;`	`FAIL: 256 vectors checked, 95 errors`
Fault removed (restored)	`PASS: 256 vectors checked, 0 errors`

The first mismatch with the fault was `a=3 b=3 expected p=9 got p=5` at 560 ns: the first vector where a full adder sees a carry-in of 1, which is exactly the term the fault removes.

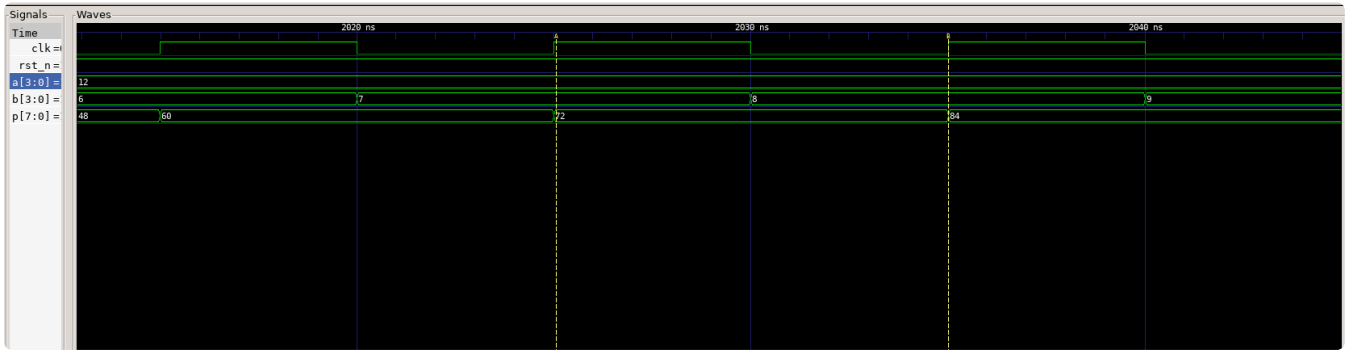


Figure 2: RTL waveform for A = 12, B = 7. Inputs change at 2020 ns (falling edge); marker A (2025 ns) is clock edge 1, where the inputs are registered; marker B (2035 ns) is edge 2, where p = 84 is registered. Latency = 2 clock cycles.

Part A: Hand-worked example

12 × 7 = 84 by hand, and the same product appears in the waveform at 2035 ns.

Roll number used: Sale Sangameshwar, 2024102017. Last digit 7, so A = 7 + 5 = 12. Since A ≠ 7, B = 7.

```
          1 1 0 0     A = 12
        x 0 1 1 1     B = 7
        ---------
          1 1 0 0     pp0 = A AND b0 (b0 = 1)
        1 1 0 0       pp1 = A AND b1 (b1 = 1), shifted 1
      1 1 0 0         pp2 = A AND b2 (b2 = 1), shifted 2
    0 0 0 0           pp3 = A AND b3 (b3 = 0), shifted 3
  ---------------
  0 1 0 1 0 1 0 0     P = 84
```

Traced through the array (bits written MSB first; full adders j = 0 to 3 listed as (sum, carry)):

Row	Inputs	Full adders j = 0..3	Outputs
0	pp0 = 1100	none	p[0] = 0, r0 = 0110
1	pp1 = 1100, r0 = 0110	(0,0) (1,0) (0,1) (0,1)	p[1] = 0, r1 = 1001
2	pp2 = 1100, r1 = 1001	(1,0) (0,0) (1,0) (0,1)	p[2] = 1, r2 = 1010
3	pp3 = 0000, r2 = 1010	(0,0) (1,0) (0,0) (1,0)	p[3] = 0, p[7:4] = 0101

Result: p = 0101 0100 = 84. Each row checks in decimal: 12 + 6 = 18 = 10010 (row 1), 12 + 9 = 21 = 10101 (row 2), 0 + 10 = 10 = 1010 (row 3).

In the waveform: the vector index is 16 × 12 + 7 = 199, so the testbench applies it at 30 + 10 × 199 = 2020 ns. It is registered at edge 1 (2025 ns) and p = 84 appears at edge 2 (2035 ns), as shown in Figure 2.

Part B: Logic synthesis

Yosys maps the design to 61 SKY130 cells with a total area of 756.976 μm². The gate-level netlist passes the same 256-vector testbench as the RTL.

`synth/synth.tcl` sets `$lib` to `$PDK_ROOT/sky130A/libs.ref/sky130_fd_sc_hd/lib/sky130_fd_sc_hd__tt_025C_1v80.lib` and runs, as `yosys <command>` lines:

1. `read_verilog rtl/full_adder.v rtl/mult_array.v`
2. `synth -top mult_array -flatten` (generic synthesis; the 12 full adders are flattened into one module)

3. `dfflibmap -liberty $lib` (flip-flops mapped to SKY130 cells)
4. `abc -liberty $lib` (logic mapped to SKY130 gates)
5. `opt_clean -purge`
6. `tee -o synth/mult_array_stat.rpt stat -liberty $lib`
7. `write_verilog -noattr synth/mult_array_netlist.v`

Run with `yosys -c synth/synth.tcl` (exit code 0; the only warnings were ABC's informational notes).

Part B results table (from `synth/mult_array_stat.rpt`):

Metric	Value
Total number of cells	61
Number of flip-flops	16 (`sky130_fd_sc_hd__dfrtp_1`)
Total cell area (μm²)	756.976
Flip-flop area (μm²)	16 × 25.024 = 400.384
Flip-flop area (% of total)	100 × 400.384 / 756.976 = 52.89 %
The 3 most-used cell types and their counts	`dfrtp_1` × 16, `nand2_1` × 7, then `and2_0` × 4 and `maj3_1` × 4 (tied)
Netlist simulation result	`PASS: 256 vectors checked, 0 errors`

The area of one `dfrtp_1` (25.024 μm²) was confirmed in the Liberty file. `dfrtp` is a D flip-flop with an active-low asynchronous reset, matching the RTL's `rst_n` behaviour.

Gate-level simulation used the command from the assignment (the PDF wraps the netlist path across two lines; it is one path):

```
CELLS=$PDK_ROOT/sky130A/libs.ref/sky130_fd_sc_hd/verilog
iverilog -g2012 -DFUNCTIONAL -DUNIT_DELAY=#1 -o gls \
    tb/tb_mult.v $CELLS/primitives.v $CELLS/sky130_fd_sc_hd.v
synth/mult_array_netlist.v
vvp gls
```

Output: `PASS: 256 vectors checked, 0 errors`, so synthesis preserved the behaviour, including the 2-cycle latency and the reset.

Report questions answered from Part B

Question 1: logic cell count. Synthesis produced 61 − 16 = 45 logic cells, not 16 + 12 = 28. After `synth -flatten` the full adders no longer exist as units, and ABC re-maps the whole array as one Boolean network into library gates. A full adder needs an XOR3 for the sum and a majority function for the carry, so it costs at least two cells (`xor3_1` + `maj3_1` , or several 2-input gates). In the other direction, ABC merges AND gates into compound cells (`a21oi` , `a22oi` , `o2111ai` , `nand3` / `nand4`), and simplifies adders with a constant input: c*i*[0] = 0 in every row and r0[3] = 0. ABC also skips SKY130's multi-output full-adder cell (`fa_1` ; its log warns of "9 multi-output cells"). The count therefore reflects the optimised logic, not the drawing.

Question 3: flip-flop share of area. Flip-flops take 400.384 of 756.976 µm², or 52.89 %, although they are only 16 of 61 cells (26 %). One `dfrtp_1` is 25.024 µm², about 6.7 times a `nand2_1` (3.754 µm²), because it contains a master and a slave latch plus reset circuitry. The register count grows linearly with the port width (2N input bits + 2N output bits = 16 here), while the array logic grows with N². For a 4-bit design the N² logic is still small, so registers dominate; at 8 or 16 bits the logic share would rise.

Commands, issues and commits

All commands ran inside the IIC-OSIC-TOOLS container from `/foss/designs` (the repository root), in this order:

```
# host (WSL 2)
docker pull hpretl/iic-osic-tools:2026.10
.dev/start_container.sh

# container
cd /foss/designs
iverilog -V | head -1; yosys -V; sta -version
echo $PDK_ROOT $PDK          # showed /foss/pdks ihp-sg13g2
sak-pdk sky130A              # now /foss/pdks sky130A

iverilog -g2012 -o sim tb/tb_mult.v rtl/full_adder.v rtl/mult_array.v
vvp sim                      # PASS
# edit full_adder.v: assign sum = a ^ b;  -> rerun: FAIL, 95 errors
# restore assign sum = a ^ b ^ cin;      -> rerun: PASS
gtkwave mult.vcd

yosys -c synth/synth.tcl
```

```
CELLS=$PDK_ROOT/sky130A/libs.ref/sky130_fd_sc_hd/verilog
iverilog -g2012 -DFUNCTIONAL -DUNIT_DELAY=#1 -o gls tb/tb_mult.v
$CELLS/primitives.v $CELLS/sky130_fd_sc_hd.v synth/mult_array_netlist.v
vvp gls                      # PASS
```

Issue	Fix
Docker Desktop was not running on Windows	Used the Docker Engine running inside WSL 2 Ubuntu
The container's default PDK was `ihp-sg13g2`	Ran `sak-pdk sky130A`, as the assignment advises
Mismatch messages printed picosecond values labelled "ns"	Added `$timeformat(-9, 0, " ns", 0)` to the testbench
`.dev/*.sh` checked out with Windows CRLF line endings, so bash rejected them	Converted to LF
WSL has about 3 GB RAM, below the recommended 8 GB	Enough for Parts A and B; to be raised in `.wslconfig` before the LibreLane run

Commit	Contents
`dbb2398` Part B: Yosys synthesis	`synth/synth.tcl`, netlist, stat report, `README.md`
`d2e4803` Part A: RTL and testbench	`rtl/full_adder.v`, `rtl/mult_array.v`, `tb/tb_mult.v`

4-bit Array Multiplier: Parts C and D Report

Continuation of the Parts A and B report · Palla Rithik Reddy (2024102005) and Sale Sangameshwar (2024102017)

Static timing analysis of the synthesized netlist gives T_{min} = 2.787 ns (F_{max} = 358.81 MHz) in the typical corner. LibreLane then took the RTL to a DRC- and LVS-clean GDSII on a 150 µm × 150 µm die: the flow met timing in every corner at CLOCK_PERIOD = 8.0 ns after 10 runs, with 0 DRC and 0 LVS errors.

Item	Version / value
Environment	IIC-OSIC-TOOLS `hpretl/iic-osic-tools:2026.10`, WSL 2 on Windows 11
Timing	OpenSTA 3.1.0, typical corner `sky130_fd_sc_hd__tt_025C_1v80`
Physical design	LibreLane v3.1.0.dev3 (OpenROAD, Magic, KLayout, Netgen), PDK sky130A

Part C: Static timing analysis

`sta/sta.tcl` performs the seven steps from the assignment: `read_liberty $lib`, `read_verilog synth/mult_array_netlist.v` and `link_design mult_array`, `create_clock -name clk -period $T [get_ports clk]`, a 1.0 ns input delay on every input except `clk`, a 1.0 ns output delay on every output, `report_worst_slack -max`, and `report_checks -path_delay max -digits 3 > sta/mult_array_timing.rpt`. T defaults to 10 ns and can be overridden with the environment variable `STA_PERIOD`.

```
sta sta/sta.tcl                    # worst slack max 7.213
STA_PERIOD=2.79 sta sta/sta.tcl    # worst slack max 0.003
```

T_{min} = T − WNS = 10 − 7.213 = 2.787 ns and F_{max} = 1000 / 2.787 = 358.81 MHz. Re-running at T = 2.79 ns leaves +0.003 ns of slack, so the path just fits.

Metric	Value
Worst slack at T = 10 ns (ns)	7.213
T_{min} (ns)	2.787 (2.79 rounded to 0.01 ns)
F_{max} (MHz)	358.81
Slack at T = T_{min} = 2.79 ns (ns)	0.003
Critical path start point	`_91_` = input register `a_r[1]` (dfrtp_1, CLK)
Critical path end point	`_88_` = output register `p[6]` (dfrtp_1, D)
Number of cells on the critical path	10 (2 flip-flops + 8 logic cells)
Latency (ns) = 2 × T_{min}	5.574
Throughput (million results per second)	358.81 (one result per cycle)

Cells on the critical path

Instance names in the report are Yosys-generated. They were mapped back to the RTL by reading `synth/mult_array_netlist.v`: `_91_` has D = `a[1]`, Q = `a_r[1]`, and `_88_` has D = `fa3_3.sum`, Q =

`p[6]`.

#	Instance	Cell	Delay (ns)	Arrival (ns)	Role in the array
1	`_91_`	dfrtp_1	0.398	0.398	launch flip-flop `a_r[1]`, CLK→Q
2	`_37_`	clkinv_1	0.073	0.471	inverts `a_r[1]` for the merged row-1 logic
3	`_42_`	o311ai_0	0.356	0.827	row 1: part of s1[1] (fa1_1 sum; pp0[2], pp1[1] and the fa1_0 carry folded in)
4	`_45_`	a21oi_1	0.153	0.980	row 2: c2[1] = pp2[0]·r1[0] (fa2_0 carry; a half adder since c2[0] = 0)
5	`_51_`	maj3_1	0.402	1.382	row 2: c2[2] (fa2_1 carry)
6	`_58_`	xnor3_1	0.370	1.752	row 2: s2[2] = r2[1] (fa2_2 sum)
7	`_61_`	maj3_1	0.368	2.119	row 3: c3[2] (fa3_1 carry)
8	`_63_`	maj3_1	0.354	2.474	row 3: c3[3] (fa3_2 carry)
9	`_64_`	xor3_1	0.178	2.652	row 3: s3[3] = p[6] (fa3_3 sum)
10	`_88_`	dfrtp_1	setup 0.135	2.787	capture flip-flop `p[6]`

All 8 logic cells implement full-adder logic, with the partial-product AND gates merged into them by ABC. The path crosses 7 full-adder stages (fa1_1 → fa2_0 → fa2_1 → fa2_2 → fa3_1 → fa3_2 → fa3_3), close to the estimate of 2N = 8. It is one stage shorter because the first adder in each row has a constant 0 carry-in and ABC folded row 0 and the start of row 1 into compound gates. The report's second path group is the reset recovery check (`rst_n` → `RESET_B`), with 2.018 ns of slack at 2.79 ns, so it does not limit F_{max}.

Part D: Physical design

CLOCK_PERIOD started at 1.2 × T_{min} = 1.2 × 2.787 = 3.344 ns, rounded up to the next 0.5 ns: **3.5 ns**. `pnr/config.json` contains exactly the required keys:

```
{
    "DESIGN_NAME": "mult_array",
    "VERILOG_FILES": ["dir::../rtl/full_adder.v", "dir::../rtl/mult_array.v"],
    "CLOCK_PORT": "clk",
    "CLOCK_PERIOD": 8.0,
    "FP_SIZING": "absolute",
    "DIE_AREA": [0, 0, 150, 150]
}
```

```
cd pnr
librelane config.json
```

Runs

LibreLane signs off timing in nine corners (min/nom/max wire RC × tt_025C_1v80, ss_100C_1v60, ff_n40C_1v95), and `timing__setup__ws` is the worst over all of them. The typical corner met timing at

3.5 ns, but the slow corner (1.60 V, 100 °C) did not. Following the assignment, CLOCK_PERIOD was raised by 0.5 ns per run until the worst slack was ≥ 0.

Run	Folder (RUN_2026-10-23_...)	CLOCK_PERIOD (ns)	Setup WS, all corners (ns)	Setup WS, nom_tt (ns)	DRC / LVS	Result
1	`23-04-36`	3.5	−2.546	0.431	0 / 0	setup violation (ss)
2	`23-09-59`	4.0	−2.027	0.929	0 / 0	setup violation (ss)
3	`23-15-41`	4.5	−1.527	1.429	0 / 0	setup violation (ss)
4	`23-19-59`	5.0	−1.027	1.929	0 / 0	setup violation (ss)
5	`23-24-04`	5.5	−0.527	2.429	0 / 0	setup violation (ss)
6	`23-28-24`	6.0	−0.037	2.911	0 / 0	setup violation (ss)
7	`23-32-27`	6.5	−0.205	3.111	0 / 0	setup violation (ss)
8	`23-36-21`	7.0	−0.116	3.526	0 / 0	setup violation (ss)
9	`23-40-39`	7.5	−0.138	3.722	0 / 0	setup violation (ss)
10	`23-44-39`	**8.0**	+0.362	4.222	0 / 0	Flow complete

DRC and LVS were clean in every run; only the slow-corner setup check failed. From 6.5 ns on, the slack stopped improving by 0.5 ns per step. LibreLane re-synthesizes the RTL with a timing-driven ABC script that depends on CLOCK_PERIOD, and the resizer then placed two `clkdlybuf4s25_1` buffers (`wire19`, `max_cap18`, long-wire and max-capacitance repair) on the critical path, which cost about 1.24 ns in the ss corner. Runs 9 and 10 have the same netlist (ss data arrival 7.643 ns in both), and 8.0 ns is the first period that covers it.

Results

Metric	Value
CLOCK_PERIOD used (ns)	8.0
Number of runs needed	10
Core area (μm²)	17 759.5 (die area 22 500 = 150 × 150)
Cells after synthesis (from Part B)	61
Cells after place and route	1 884 (`design__instance__count`)
Kinds of cells added	decap (1 299), fill (254), tap (245), port buffers (17 `buf_2`), repair buffers (4 `clkdlybuf4s*`), clock buffers (3 `clkbuf_16`)
Wirelength (μm)	2 518 (detailed route)
Setup slack (ns)	+0.362 (worst corner `max_ss_100C_1v60`); +4.222 in `nom_tt_025C_1v80`
Post-route F_{max} (MHz)	1000 / (8.0 − 0.362) = **130.92** (all corners); 1000 / (8.0 − 4.222) = 264.72 (typical corner)
F_{max} from Part C (MHz)	358.81 (typical corner, ideal clock, no wires)
DRC errors	**0** (Magic 0, KLayout 0)
LVS errors	**0**

Post-route F_{max} = 1000 / (CLOCK_PERIOD − setup slack). With the all-corner worst slack this is 130.92 MHz. The like-for-like comparison with Part C, which used the typical corner only, is 264.72 MHz against 358.81 MHz. Hold slack is positive in every corner (worst +0.141 ns), there are no antenna violations, and the only remaining checker warning is max slew in the ss corners (a warning, not a signoff error).

Cells added by the flow

1 884 − 61 = 1 823 cells were added, counted by cell name in the final DEF:

Kind	Cells	Count	Why it is needed
Logic from LibreLane's own synthesis	dfrtp_1 ×16 + 46 gates	62	the multiplier itself (Part B gave 61 with a different ABC script)
Decap	decap_3/4/6/8/12	1 299	on-chip capacitance between VPWR and VGND that steadies the supply when cells switch
Fill	fill_1, fill_2	254	close the remaining gaps in the rows so wells, implants and power rails stay continuous
Tap	tapvpwrvgnd_1	245	tie the n-well to VPWR and the substrate to VGND at regular spacing, preventing latch-up
Port buffers	buf_2 ×17 (input1–9, output10–17)	17	isolate the 9 input and 8 output pins from the core and drive the output load
Repair buffers	clkdlybuf4s50_1 ×2, clkdlybuf4s25_1 ×2	4	fix fanout, long-wire and max-capacitance violations
Clock tree	clkbuf_16 ×3	3	CTS: a root and two leaf buffers deliver the clock to the 16 flip-flops with low skew

Most of the added cells are decap and fill, because the logic covers only 7.1 % of the 17 759.5 µm² core.

Layout

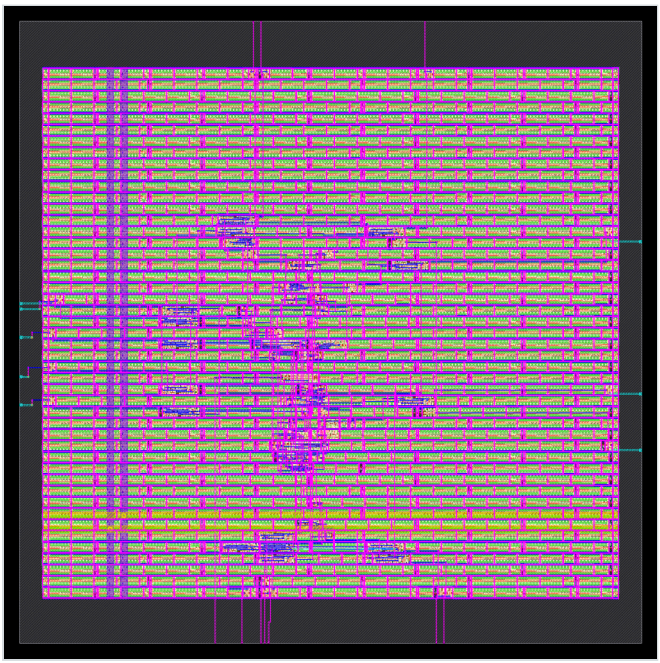


Figure 3: Whole 150 µm × 150 µm die in KLayout. The core holds standard-cell rows with the met4/met5 power straps; I/O pins sit on the die edges. Most of the rows are fill and tap cells; the 16 flip-flops and the array logic sit near the centre.

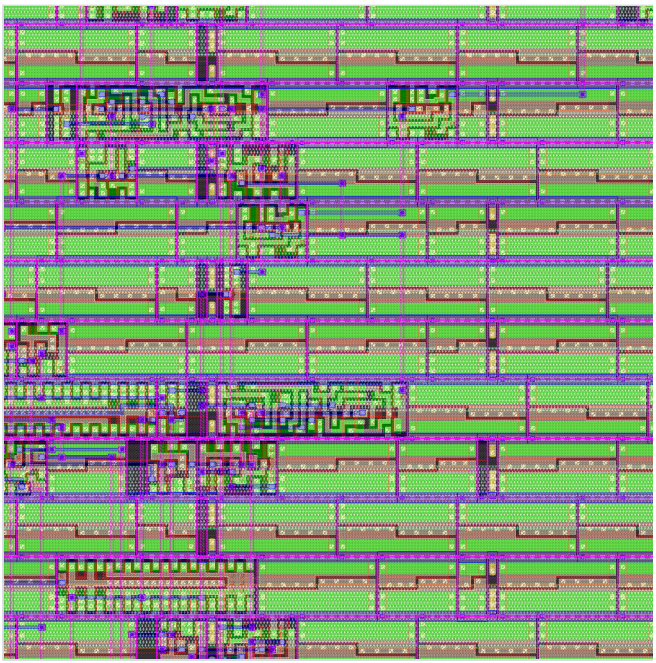


Figure 4: 30 µm × 30 µm zoom at the centre of the core: horizontal standard-cell rows with met1 power rails, logic cells between fill/decap cells, and met1/met2 signal wires with vias.

Answers to the report questions

1. Logic cells after synthesis

Synthesis produced 61 cells, of which 16 are flip-flops, so 61 − 16 = 45 cells implement the logic, not 16 + 12 = 28. With `synth -flatten` the full adders stop existing as units and ABC maps the whole array as one Boolean network. A full adder needs an XOR3 for the sum and a majority function for the carry, so it costs at least two library cells (for example `xor3_1` + `maj3_1`). In the other direction, ABC absorbs AND gates into compound cells (`a21oi`, `a22oi`, `o2111ai`, `nand3` / `nand4`) and simplifies the adders with a constant 0 input ($c_i[0] = 0$ in every row and r0[3] = 0). The count reflects the optimised logic, not the schematic. (Also answered in the Parts A and B section.)

2. Critical path

The critical path runs from input register `a_r[1]` (`_91_`) to output register `p[6]` (`_88_`). It contains 10 cells: the launch flip-flop, 8 logic cells (clkinv, o311ai, a21oi, maj3, xnor3, maj3, maj3, xor3) and the capture flip-flop. The data arrives at 2.652 ns and the setup time is 0.135 ns, giving T_{min} = 2.787 ns. The path ripples through 7 full-adder stages (fa1_1 → fa2_0 → fa2_1 → fa2_2 → fa3_1 → fa3_2 → fa3_3): down the rows and then along the carry chain of row 3. That is close to the 2N = 8 estimate. It is one stage shorter because the first adder of each row has a 0 carry-in, so it reduces to a half adder that ABC merges with the AND gates.

3. Flip-flop share of the area

The 16 `dfrtp_1` flip-flops take 16 × 25.024 = 400.384 µm² of 756.976 µm², or 52.89 %, although they are only 16 of 61 cells (26 %). One `dfrtp_1` is about 6.7 times a `nand2_1` (3.754 µm²) because it contains a master latch, a slave latch, clock inverters and reset circuitry. The register count grows linearly with N (2N input bits + 2N output bits = 16), while the array logic grows with N². At N = 4 the N² logic is still small, so the registers dominate; at 8 or 16 bits the logic share would rise. (Also answered in the Parts A and B section.)

4. Post-route F_{max} and added cells

Post-route F_{max} is 130.92 MHz over all corners and 264.72 MHz in the typical corner, against 358.81 MHz from Part C. Comparing the typical-corner critical path of the final run with Part C shows where the time goes. Part C assumed an ideal clock and no wires. After routing, the extracted wire resistance and capacitance load every gate: the launch flip-flop's clock-to-Q rises from 0.398 ns to 0.525 ns. The real clock tree adds about 0.32 ns of insertion delay with some skew between the two leaf buffers, and LibreLane adds 0.25 ns of clock uncertainty. The cells are spread over a core that is only 7 % utilised, so some nets are long, and the resizer put two delay buffers (`wire19`, `max_cap18`) on the critical path, 0.61 ns in tt. LibreLane also re-synthesizes the RTL with its own script, giving a slightly different netlist. The data arrives at 3.715 ns instead of 2.652 ns. The all-corner figure is lower again because it is set by the slow corner (1.60 V, 100 °C), where the same path arrives at 7.643 ns.

The flow added 1 299 decap cells (supply decoupling against IR drop and switching noise), 254 fill cells (continuous wells, implants and rails, needed to pass DRC), 245 tap cells (well and substrate ties against latch-up), 17 `buf_2` port buffers (isolate the pins and drive the external load), 4 repair buffers (fanout, long-wire and max-capacitance repair) and 3 `clkbuf_16` clock-tree buffers (drive all 16 flip-flops with low skew).

5. DRC and LVS

DRC (design rule check) verifies that the drawn geometry obeys the foundry's manufacturing rules: minimum widths and spacings of every layer, enclosure of vias and contacts, minimum areas, density and antenna limits. A violation means the masks may not print as drawn, giving shorts, opens or low yield. LVS (layout versus schematic) extracts the transistors and their connections from the layout and compares them with the netlist, checking that every device and net matches and no net is shorted or left open. A violation means the chip, even if manufacturable, is not the circuit that was verified in simulation and STA. Both must pass: DRC says the layout can be built, LVS says it is the right circuit. Our final run reports 0 Magic DRC errors, 0 KLayout DRC errors and 0 LVS errors (Netgen: 0 device, net and pin mismatches).

6. Scaling to 8 bits

For N = 4 the array uses $N^2 = 16$ AND gates and $N(N - 1) = 4 \times 3 = 12$ full adders. For N = 8 it would use $8^2 = 64$ AND gates and $8 \times 7 = 56$ full adders, about 4.7 times as many (the adder count grows roughly with N^2). The registers would grow from 16 to 2N + 2N = 32 flip-flops.

The critical path is about 2N full-adder delays: $2 \times 4 = 8$ for N = 4 and $2 \times 8 = 16$ for N = 8, so the logic delay roughly doubles. From Part C, the logic part of our path is 2.652 – 0.398 = 2.254 ns for an estimated 8 stages, about 0.28 ns per full-adder delay. For N = 8: $16 \times 0.28 = 4.51$ ns of logic, plus 0.398 ns clock-to-Q and 0.135 ns setup, gives $T_{min} \approx 5.04$ ns and $F_{max} \approx 1000 / 5.04 \approx 198$ MHz, compared with 358.81 MHz at N = 4. Area grows quadratically while delay grows only linearly, which is the usual trade-off of the array multiplier.

Commands, issues and commits

All commands ran inside the IIC-OSIC-TOOLS container from `/foss/designs` (the repository root), after `sak-pdk sky130A`:

```
# Part C
sta sta/sta.tcl                       # T = 10 ns: worst slack 7.213
STA_PERIOD=2.79 sta sta/sta.tcl       # T = Tmin:  worst slack 0.003

# Part D (CLOCK_PERIOD edited in pnr/config.json before each run:
#   3.5, 4.0, 4.5, 5.0, 5.5, 6.0, 6.5, 7.0, 7.5, 8.0)
cd pnr
librelane config.json
cp runs/RUN_2026-10-23_23-44-39/final/metrics.json .
cp runs/RUN_2026-10-23_23-44-39/final/gds/mult_array.gds .
klayout mult_array.gds
```

Issue	Fix
`sta sta/sta.tcl` stayed in the interactive OpenSTA shell after the script	Added `exit` at the end of the script
`report_worst_slack -max` printed only 2 decimals (7.21)	Added `-digits 3` so T_{min} is exact to 0.001 ns
The timing report names cells `_91_`, `_88_` ...	Mapped them to `a_r[1]`, `p[6]` through their D/Q nets in the netlist
Setup violations at 3.5 ns, only in the ss_100C_1v60 corners	Raised CLOCK_PERIOD by 0.5 ns per run; passed at 8.0 ns in run 10
WSL has about 3 GB RAM, below the recommended 8 GB	Enough for this design: each LibreLane run took 4–5 minutes

Commit	Contents
Part C: OpenSTA timing analysis	`sta/sta.tcl`, `sta/mult_array_timing.rpt`, `README.md`
Part D: LibreLane RTL-to-GDSII	`pnr/config.json`, `pnr/metrics.json`, `pnr/mult_array.gds`, `README.md`
Report: Parts C and D, answers to questions 1–6	`report/report.pdf`, `README.md`